



RISEUP Internship 2024 - Internship Report

Name of the Intern: Karan Singh

Name of the Supervisor: Dr. Sidharth Maheshwari

Project Title: TinyML Application for the Edge

Project Details: The project focuses on generating sine wave signals using an Arduino Nano 33 BLE Lite microcontroller, a DAC DFR0972 module, and an oscilloscope. The objective is to leverage TinyML concepts to generate sine wave data, visualize it using a serial plotter, and then output the wave through the DAC to an oscilloscope using I2C communication. This project integrates basic concepts of signal processing, microcontroller programming, and data transfer protocols, providing a practical application of embedded systems and TinyML in a real-world scenario.

Verified by: Yukti Gupta

Faculty Name: Dr. Sidharth Maheshwari

Date: 31st of July 2024



Abstract

This project explores the application of Tiny Machine Learning (TinyML) on edge devices using the Arduino Nano 33 BLE Lite and DAC DFR0972. The primary objective is to generate and visualize sine wave data on an oscilloscope, leveraging the capabilities of TinyML and hardware interfacing. The study begins with data generation using a mathematical sine function $Y=\sin(x)$ visualized through a serial plotter. This data is then transmitted to a Digital-to-Analog Converter (DAC) using I2C communication protocols, subsequently displayed on an oscilloscope. The project emphasizes the integration of machine learning techniques with edge computing hardware, showcasing practical applications such as sensor data processing and real-time analytics. The findings demonstrate the feasibility and effectiveness of deploying TinyML models in resource-constrained environments, paving the way for advanced IoT applications.



Acknowledgement

I would like to express my deepest gratitude to my professor **Dr.Sidharth Maheshwari** at IIT Jammu for their invaluable guidance and support throughout this project. Their insights and expertise were instrumental in shaping the direction and outcome of this research.

I am also thankful to the Department of Computer Science and Engineering at IIT Jammu for providing me with the opportunity and resources to pursue this internship. The Arduino TinyML kit and DAC module were crucial in developing and implementing the project.

Additionally, I appreciate the wealth of knowledge shared by the creators of the book 'TinyML' by Pete Warden and Daniel Situnayake, which served as a foundational resource for understanding and applying TinyML concepts.

Lastly, I extend my gratitude to my friends for their unwavering support and encouragement throughout this journey.

Table of Contents

S.No.	Title	Page No.
1.	Introduction	5
2.	Literature Review	6-8
3.	Methodology	9-38
4.	Results and Discussion	39-40
5.	Conclusion	41-42
6.	Project Summary	43-44
7.	References	45

1. Introduction

Background:

Sine waves are fundamental in electronics and signal processing, representing periodic oscillations found in various natural and technological phenomena. They are crucial in understanding concepts such as alternating current (AC), signal modulation, and waveform analysis. The smooth, repetitive oscillations of sine waves are pivotal in the analysis and synthesis of signals in communication systems, audio processing, and control systems.

The emergence of Tiny Machine Learning (TinyML) brings the power of machine learning to embedded systems, enabling intelligent data processing on low-power devices. TinyML applications are especially relevant in scenarios where real-time data processing is required without the reliance on cloud computing. By leveraging TinyML, embedded systems can perform tasks like anomaly detection, predictive maintenance, and pattern recognition directly on-device, making them highly efficient and responsive.

Objectives:

The primary objectives of this project are:

1. To generate synthetic sine wave data using mathematical functions and visualize it in real-time.
2. To transmit this data to a Digital-to-Analog Converter (DAC) for conversion and display on an oscilloscope.
3. To demonstrate the integration of TinyML techniques in processing and analyzing the generated data.
4. To showcase the practical applications of TinyML in enhancing the capabilities of embedded systems, specifically in signal processing and real-time data analysis.

Scope:

The scope of this project includes:

- Utilizing the Arduino Nano 33 BLE Lite and DAC DFR0972 for data generation and visualization.
- Implementing and demonstrating basic TinyML concepts to process sine wave data.
- Focusing on the hardware and software integration necessary for the project, including I2C communication and data conversion techniques.

2. Literature Review

Related Work:

The generation and analysis of sine waves have long been fundamental aspects of electronics and signal processing. Numerous studies and projects have explored various methods for generating and manipulating sine waves, often using microcontrollers and digital-to-analog converters (DACs). For instance, the use of Arduino microcontrollers in generating waveforms has been widely documented, highlighting their versatility in educational and prototyping contexts.

In recent years, the advent of Tiny Machine Learning (TinyML) has expanded the capabilities of microcontrollers by enabling on-device data processing and machine learning. TinyML allows for the deployment of lightweight machine learning models on microcontrollers, facilitating applications such as sensor data analysis, predictive maintenance, and real-time decision-making. Notable projects have demonstrated the use of TinyML in gesture recognition, anomaly detection, and environmental monitoring, often leveraging platforms like EdgeImpulse for model development and deployment.

Technical Concepts:

Sine Wave Generation: Sine waves are mathematical curves that describe a smooth, periodic oscillation. They are defined by the function $y = \sin(x)$, where x is the angle in radians. In electronics, sine waves are essential for representing AC signals and analyzing the behavior of systems under oscillatory conditions. Generating sine waves digitally often involves using a microcontroller to calculate the wave's values at discrete intervals, which are then converted into analog signals.

Digital-to-Analog Converter (DAC): A DAC is a device that converts digital data, typically binary, into an analog signal. This conversion is crucial in applications where digital systems interact with the analog world, such as in audio playback, signal synthesis, and instrumentation. The quality of a DAC is often characterized by its resolution and conversion speed, which determine the accuracy and fidelity of the analog output.

I2C Communication: I2C (Inter-Integrated Circuit) is a synchronous serial communication bus developed by Philips Semiconductors (now NXP). It's commonly used for connecting low-speed devices like sensors, EEPROMs, and microcontrollers.

1. I2C Bus Architecture

1.1. Components

- **Master:** Initiates communication, controls the clock, and sends commands.
- **Slave:** Responds to the master's commands. Each slave has a unique address.
- **Bus Lines:**
 - **SDA (Serial Data Line):** Carries data.
 - **SCL (Serial Clock Line):** Carries the clock signal.

2. I2C Communication Protocol

2.1. Data Transfer

- **Start Condition:** Initiated by the master to signal the beginning of communication. This is done by pulling SDA low while SCL is high.
- **Stop Condition:** Signaled by the master to end communication. This is done by pulling SDA high while SCL is high.

2.2. Addressing

- **Address Frame:** Each slave device has a unique 7-bit or 10-bit address. The master sends the address to select the slave.
- **Read/Write Bit:** Following the address, a bit is sent to indicate if the master wants to read from (1) or write to (0) the slave.

2.3. Data Frames

- **Data Byte:** Each data byte is 8 bits long. After each byte, an acknowledgment (ACK) or non-acknowledgment (NACK) bit is sent by the receiver.

2.4. Acknowledgment

- **ACK:** Receiver pulls SDA low to acknowledge receipt of a byte.
- **NACK:** Receiver leaves SDA high to indicate no acknowledgment.

3. I2C Operations

3.1. Master Transmit

1. **Start Condition:** Master sends a start condition.
2. **Address Byte:** Master sends the 7-bit address and read/write bit.
3. **Data Byte(s):** Master sends data bytes.
4. **Stop Condition:** Master sends a stop condition to end communication.

3.2. Master Receive

1. **Start Condition:** Master sends a start condition.
2. **Address Byte:** Master sends the 7-bit address with read/write bit set to read.
3. **Data Byte(s):** Slave sends data bytes.
4. **ACK/NACK:** Master acknowledges receipt of each byte.
5. **Stop Condition:** Master sends a stop condition.

3.3. Slave Operations

- **Slave Transmit:** Slave receives address and data, then sends data bytes to the master.
- **Slave Receive:** Slave receives data bytes from the master.

Tiny Machine Learning (TinyML): TinyML refers to the application of machine learning techniques on low-power, resource-constrained devices, such as microcontrollers. This field focuses on optimizing machine learning models to run efficiently on hardware with limited computational and memory resources. TinyML enables the deployment of intelligent functions in edge devices, reducing the need for data transmission to central servers and allowing for real-time data processing and decision-making.

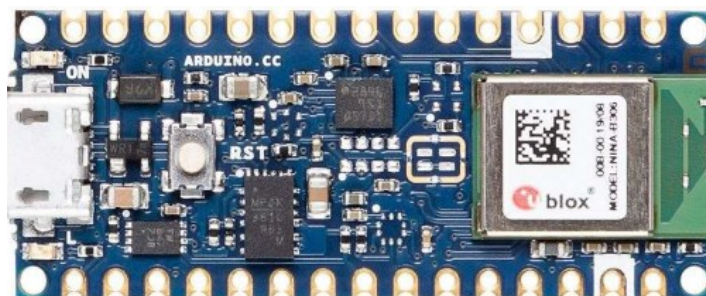
3. Methodology

Hardware and Software Used

Hardware Components:

Arduino Nano 33 BLE Lite

The **Arduino Nano 33 BLE Lite** is a compact and powerful microcontroller board designed for a wide range of applications, including Bluetooth Low Energy (BLE) projects. It offers a combination of performance, connectivity, and flexibility in a small form factor.



1. Key Features

1.1. Microcontroller

- **Chip:** Nordic Semiconductor nRF52840
- **Core:** ARM Cortex-M4
- **Clock Speed:** 64 MHz

1.2. Memory

- **Flash Memory:** 256 KB
- **RAM:** 32 KB

1.3. Connectivity



विद्याधनं सर्वधनं प्रधानम्

IIT JAMMU

- **Bluetooth Low Energy (BLE):** Supports Bluetooth 5.0, allowing for wireless communication with other BLE devices.
- **USB:** For programming and serial communication.

1.4. Input/Output

- **Digital I/O Pins:** 14 digital pins, which can be used for various functions such as GPIO, PWM, etc.
- **Analog Input Pins:** 8 analog input pins with 12-bit resolution.
- **PWM Channels:** Support for Pulse Width Modulation on certain digital pins.

1.5. Sensors

- **IMU (Inertial Measurement Unit):** Integrated 9-axis IMU sensor (LSM6DS3) for motion sensing, including accelerometer and gyroscope.

1.6. Power Supply

- **Operating Voltage:** 3.3V
- **Power Supply Options:** Can be powered via USB or external power supply.

1.7. Form Factor

- **Size:** 45 mm x 18 mm
- **Mounting:** Compatible with standard Arduino Nano form factor.

2. Key Specifications

- **Operating Voltage:** 3.3V
- **Input Voltage (via USB):** 5V
- **Current Draw:** ~6 mA (average), varies with activity and peripherals.
- **Communication Protocols:** UART, SPI, I2C, GPIO, PWM, ADC.
- **LED Indicators:** On-board LEDs for status indication.

3. Programming and Development

3.1. IDE Compatibility

- **Arduino IDE:** Supports programming using the Arduino IDE with the appropriate board package installed.
- **PlatformIO:** Can be used with PlatformIO for more advanced development.

3.2. Libraries and Examples



विद्याधनं सर्वधनं प्रधानम्

IIT JAMMU

- **Arduino Libraries:** Libraries are available for various sensors and BLE functionalities. Examples include BLE libraries for communication and sensor libraries for the IMU.

3.3. Debugging

- **Serial Monitor:** Use the Serial Monitor in the Arduino IDE for debugging and communication with the board.

4. Applications

4.1. Bluetooth Low Energy Projects

- **Wearables:** Ideal for creating wearable devices with BLE capabilities.
- **Remote Controls:** Can be used for building remote controls or other wireless communication devices.

4.2. Sensor Projects

- **Motion Detection:** Leverage the built-in IMU for motion tracking, orientation sensing, and gesture recognition.
- **Data Logging:** Collect and process data from various sensors.

4.3. IoT Projects

- **Wireless Communication:** Integrate with other IoT devices and cloud services via BLE for various Internet of Things applications.

4.4. Prototyping

- **Rapid Prototyping:** Suitable for rapid prototyping of new ideas and concepts due to its small size and versatility.

5. Getting Started

5.1. Installation

- **Arduino IDE:** Install the Arduino IDE and add the board via the Board Manager by searching for “Arduino Nano 33 BLE.”
- **Drivers:** Ensure that any necessary drivers are installed for USB communication.

5.2. Basic Setup

- **Blink Example:** Upload a simple blink sketch to test the board and familiarize yourself with the programming process.



विद्याधनं सर्वधनं प्रधानम्

IIT JAMMU

- **BLE Example:** Use example sketches to test Bluetooth communication and sensor functionality.

6. Advanced Features

6.1. Low Power Modes

- **Deep Sleep:** Supports low-power sleep modes to conserve energy during idle periods.

6.2. Security

- **Secure BLE:** Supports secure BLE communication features for encryption and authentication.

6.3. Hardware Interfaces

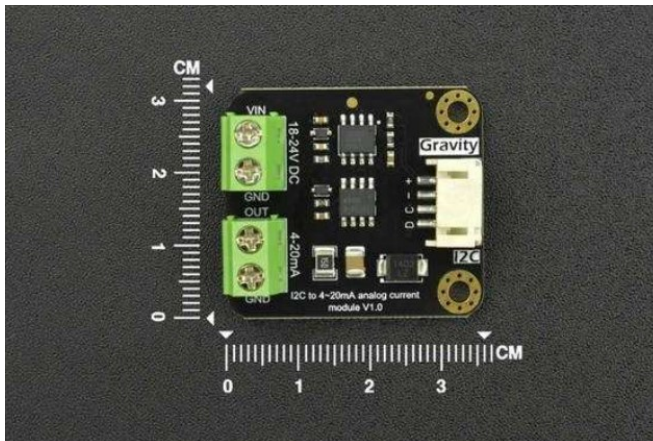
- **SPI/I2C/UART:** Use these interfaces to connect external peripherals and sensors.

7. Resources

- **Datasheet:** Comprehensive datasheet available from the Arduino website or Nordic Semiconductor's site for detailed technical specifications.
- **Arduino Documentation:** Official Arduino documentation provides detailed guides and examples for using the Nano 33 BLE Lite.
- **Community Support:** Engage with the Arduino community for support, tutorials, and shared projects.
-

DAC DFR0972

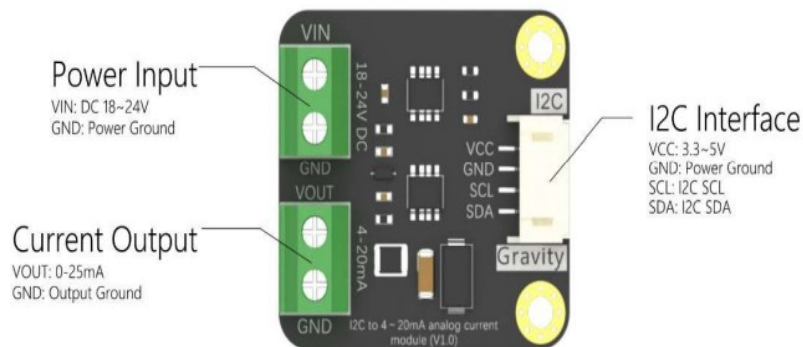
The **DAC DFR0972** is a Digital-to-Analog Converter (DAC) module designed for precise analog signal output. It is typically used in projects that require the conversion of digital signals from a microcontroller into analog signals, which can then be used to control various analog devices or to interface with analog systems.





विद्याधनं सर्वधनं प्रधानम्

IIT JAMMU



1. Key Features

1.1. Digital-to-Analog Conversion

- **Resolution:** 12-bit resolution, providing a high level of precision for converting digital values into analog signals.

1.2. Output

- **Output Range:** Typically 0 to 3.3V, but this can vary based on the power supply and configuration.
- **Output Type:** Voltage output that can be used to drive analog devices.

1.3. Communication Protocol

- **Interface:** I2C (Inter-Integrated Circuit) communication for interfacing with microcontrollers and other digital devices.

1.4. Power Supply

- **Operating Voltage:** Usually 3.3V or 5V, depending on the specific module version and design.

2. Key Specifications

- **Resolution:** 12 bits (4096 levels)
- **Output Voltage Range:** 0V to 3.3V (or other specified ranges based on power supply)
- **I2C Address:** Configurable I2C address for communication with microcontrollers.
- **Power Supply:** 3.3V or 5V

3. Applications

3.1. Signal Generation

- **Analog Signal Output:** Generate analog signals from digital data for various applications such as waveform generation, audio signal processing, and control systems.

3.2. Control Systems

- **Analog Control:** Control analog devices such as motors, LEDs, or other hardware that requires analog input.

4. Communication and Integration

4.1. I2C Communication

- **Addressing:** The DAC DFR0972 uses I2C communication, which allows it to be addressed and controlled by a microcontroller or other I2C master devices.
- **Data Transmission:** Send digital data to the DAC via I2C commands, which the DAC then converts into an analog voltage.

4.2. Integration with Microcontrollers

- **Arduino:** Easily interfaced with Arduino boards using I2C libraries and example code.
- **Other Microcontrollers:** Can be used with other microcontrollers that support I2C communication.

Software Tools:

1. Arduino IDE

- **Description:** The Arduino Integrated Development Environment (IDE) is a software application used for writing, compiling, and uploading code to Arduino microcontrollers. It provides a user-friendly interface and supports various programming languages, including C++ and Arduino's simplified language.
- **Key Features:**
 - Code editor with syntax highlighting and autocompletion
 - Built-in libraries and examples
 - Serial monitor for debugging
 - Support for a wide range of Arduino boards and shields

2. Libraries

- **Arduino Wire Library:** A standard library used for I2C communication in Arduino sketches. It provides functions for interfacing with I2C devices like the DAC DFR0972, allowing for data transmission and reception over the I2C bus.
- **Arduino DAC Library:** This library may be used to simplify the control and configuration of the DAC DFR0972. It includes functions for setting the DAC output and managing communication parameters.
- **Math Libraries:** Libraries or functions for generating sine wave values. This might include mathematical functions for calculating sine wave data or for handling floating-point arithmetic.

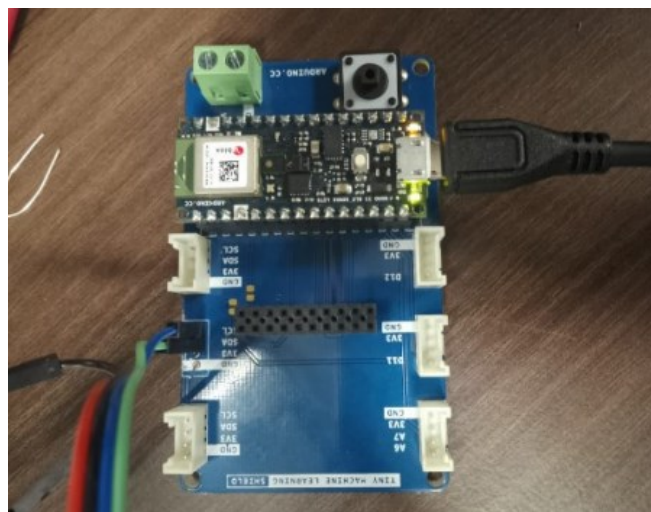
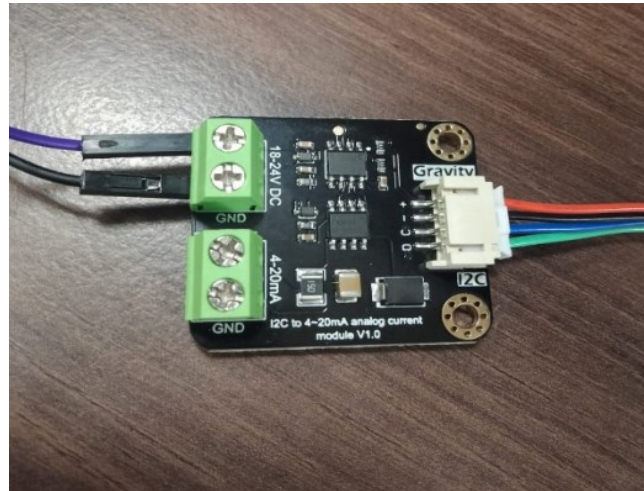
Schematic Diagram:

Components:

1. Arduino Nano 33 BLE Lite
2. DAC DFR0972
3. Oscilloscope
4. Connecting Wires

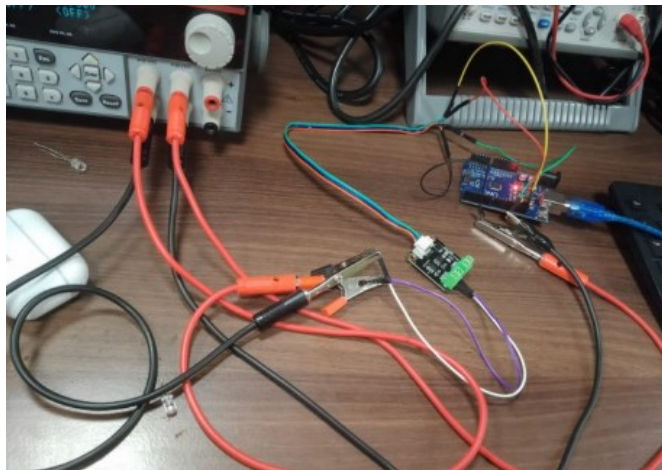
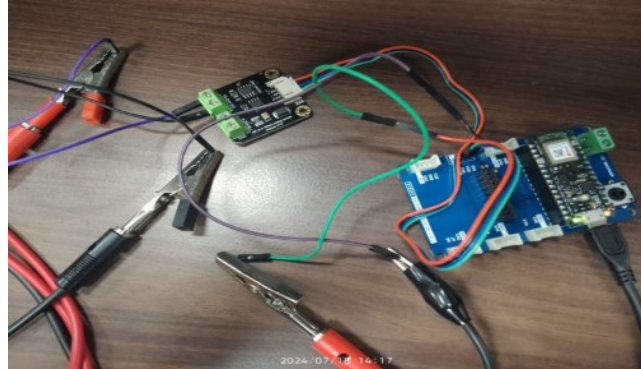
Connections:

1. **Arduino Nano 33 BLE Lite to DAC DFR0972:**
 - SDA (Data Line) on Arduino → SDA on DAC
 - SCL (Clock Line) on Arduino → SCL on DAC
 - GND on Arduino → GND on DAC
 - 3.3V or 5V on Arduino → VCC on DAC



2. DAC DFR0972 to Oscilloscope:

- Vout (Analog Output) on DAC → Channel 1 Input on Oscilloscope
- GND on DAC → GND on Oscilloscope



Building Our Model

We're now going to walk through the process of building, training, and converting our model. We include all of the code notebook. but you can also follow along in Colab and run the code as you go.

Configure Defaults

```
# Define paths to model files
import os
MODELS_DIR = 'models/'
if not os.path.exists(MODELS_DIR):
    os.mkdir(MODELS_DIR)
MODEL_TF = MODELS_DIR + 'model'
MODEL_NO_QUANT_TFLITE = MODELS_DIR + 'model_no_quant.tflite'
MODEL_TFLITE = MODELS_DIR + 'model.tflite'
MODEL_TFLITE_MICRO = MODELS_DIR + 'model.cc'
```

Setup Environment

Install Dependencies

```
! pip install tensorflow==2.4.0
```

Import Dependencies

```
# TensorFlow is an open source machine learning library
import tensorflow as tf

# Keras is TensorFlow's high-level API for deep learning
from tensorflow import keras
# Numpy is a math library
import numpy as np
# Pandas is a data manipulation library
import pandas as pd
# Matplotlib is a graphing library
import matplotlib.pyplot as plt
# Math is Python's math library
import math
```



विद्याधनं सर्वधनं प्रधानम्

IIT JAMMU

```
# Set seed for experiment reproducibility
seed = 1
np.random.seed(seed)
tf.random.set_seed(seed)
```

1. Generate Data

The code in the following cell will generate a set of random x values, calculate their sine values, and display them on a graph.

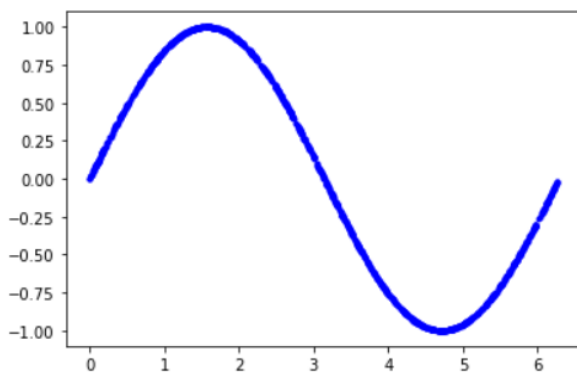
```
# Number of sample datapoints
SAMPLES = 1000

# Generate a uniformly distributed set of random numbers in the range
from
# 0 to  $2\pi$ , which covers a complete sine wave oscillation
x_values = np.random.uniform(
    low=0, high=2*math.pi, size=SAMPLES).astype(np.float32)

# Shuffle the values to guarantee they're not in order
np.random.shuffle(x_values)

# Calculate the corresponding sine values
y_values = np.sin(x_values).astype(np.float32)

# Plot our data. The 'b.' argument tells the library to print blue
dots.
plt.plot(x_values, y_values, 'b.')
plt.show()
```



2. Add Noise

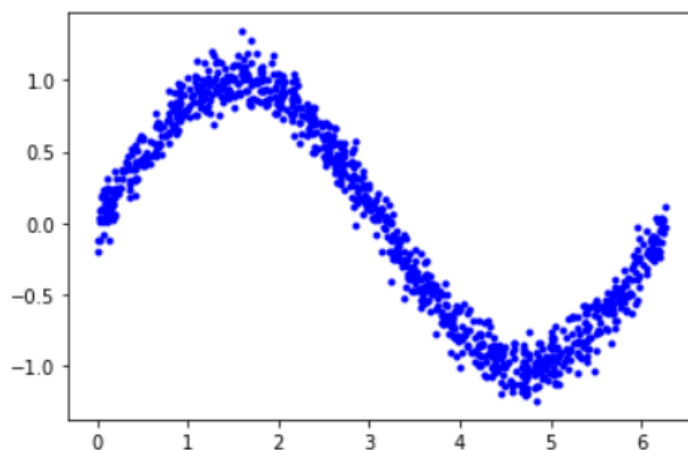
Since it was generated directly by the sine function, our data fits a nice, smooth curve.

However, machine learning models are good at extracting underlying meaning from messy, real world data. To demonstrate this, we can add some noise to our data to approximate something more life-like.

In the following cell, we'll add some random noise to each value, then draw a new graph:

```
# Add a small random number to each y value
y_values += 0.1 * np.random.randn(*y_values.shape)

# Plot our data
plt.plot(x_values, y_values, 'b.')
```



3. Split the Data

We now have a noisy dataset that approximates real world data. We'll be using this to train our model.

To evaluate the accuracy of the model we train, we'll need to compare its predictions to real data and check how well they match up. This evaluation happens during training (where it is referred to as validation) and after training (referred to as testing) It's important in both cases that we use fresh data that was not already used to train the model.

The data is split as follows:

1. Training: 60%
2. Validation: 20%
3. Testing: 20%

The following code will split our data and then plots each set as a different color:



विद्याधनं सर्वधनं प्रधानम्

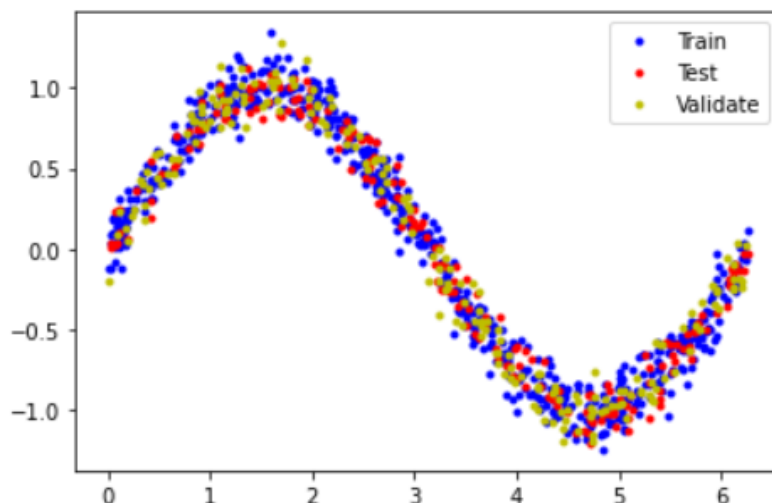
IIT JAMMU

```
# We'll use 60% of our data for training and 20% for testing. The
remaining 20%
# will be used for validation. Calculate the indices of each section.
TRAIN_SPLIT = int(0.6 * SAMPLES)
TEST_SPLIT = int(0.2 * SAMPLES + TRAIN_SPLIT)

# Use np.split to chop our data into three parts.
# The second argument to np.split is an array of indices where the data
will be
# split. We provide two indices, so the data will be divided into three
chunks.
x_train, x_test, x_validate = np.split(x_values, [TRAIN_SPLIT,
TEST_SPLIT])
y_train, y_test, y_validate = np.split(y_values, [TRAIN_SPLIT,
TEST_SPLIT])

# Double check that our splits add up correctly
assert (x_train.size + x_validate.size + x_test.size) == SAMPLES

# Plot the data in each partition in different colors:
plt.plot(x_train, y_train, 'b.', label="Train")
plt.plot(x_test, y_test, 'r.', label="Test")
plt.plot(x_validate, y_validate, 'y.', label="Validate")
plt.legend()
plt.show()
```



Training

1. Design the Model

We're going to build a simple neural network model that will take an input value (in this case, x) and use it to predict a numeric output value (the sine of x). This type of problem is called a *regression*. It will use *layers* of *neurons* to attempt to learn any patterns underlying the training data, so it can make predictions.

To begin with, we'll define two layers. The first layer takes a single input (our x value) and runs it through 8 neurons. Based on this input, each neuron will become *activated* to a certain degree based on its internal state (its *weight* and *bias* values). A neuron's degree of activation is expressed as a number.

The activation numbers from our first layer will be fed as inputs to our second layer, which is a single neuron. It will apply its own weights and bias to these inputs and calculate its own activation, which will be output as our y value.

Note: To learn more about how neural networks function, you can explore the [Learn TensorFlow](#) codelabs.

The code in the following cell defines our model using [Keras](#), TensorFlow's high-level API for creating deep learning networks. Once the network is defined, we *compile* it, specifying parameters that determine how it will be trained:

```
# We'll use Keras to create a simple model architecture
model_1 = tf.keras.Sequential()

# First layer takes a scalar input and feeds it through 8 "neurons".
# The
# neurons decide whether to activate based on the 'relu' activation
# function.
model_1.add(keras.layers.Dense(8, activation='relu', input_shape=(1,)))

# Final layer is a single neuron, since we want to output a single
# value
model_1.add(keras.layers.Dense(1))

# Compile the model using the standard 'adam' optimizer and the mean
# squared error or 'mse' loss function for regression.
model_1.compile(optimizer='adam', loss='mse', metrics=['mae'])
```


2. Train the Model

Once we've defined the model, we can use our data to *train* it. Training involves passing an x value into the neural network, checking how far the network's output deviates from the expected y value, and adjusting the neurons' weights and biases so that the output is more likely to be correct the next time.

Training runs this process on the full dataset multiple times, and each full run-through is known as an *epoch*. The number of epochs to run during training is a parameter we can set.

During each epoch, data is run through the network in multiple *batches*. Each batch, several pieces of data are passed into the network, producing output values. These outputs' correctness is measured in aggregate and the network's weights and biases are adjusted accordingly, once per batch. The *batch size* is also a parameter we can set.

The code in the following cell uses the x and y values from our training data to train the model. It runs for 500 *epochs*, with 64 pieces of data in each *batch*. We also pass in some data for *validation*. As you will see when you run the cell, training can take a while to complete:

```
# Train the model on our training data while validating on our
validation set
history_1 = model_1.fit(x_train, y_train, epochs=500, batch_size=64,
                        validation_data=(x_validate, y_validate))
```

```
Epoch 1/500
10/10 [=====] - 1s 47ms/step - loss: 0.7289 - mae: 0.7120 - val_loss: 0.6401 - val_mae: 0.6504
Epoch 2/500
10/10 [=====] - 0s 6ms/step - loss: 0.6329 - mae: 0.6488 - val_loss: 0.5587 - val_mae: 0.6031
Epoch 3/500
10/10 [=====] - 0s 6ms/step - loss: 0.5201 - mae: 0.5735 - val_loss: 0.5014 - val_mae: 0.5763
Epoch 4/500
10/10 [=====] - 0s 6ms/step - loss: 0.5057 - mae: 0.5760 - val_loss: 0.4632 - val_mae: 0.5615
Epoch 5/500
10/10 [=====] - 0s 5ms/step - loss: 0.4502 - mae: 0.5459 - val_loss: 0.4386 - val_mae: 0.5536
Epoch 6/500
10/10 [=====] - 0s 6ms/step - loss: 0.4168 - mae: 0.5332 - val_loss: 0.4227 - val_mae: 0.5490
Epoch 7/500
10/10 [=====] - 0s 6ms/step - loss: 0.4211 - mae: 0.5341 - val_loss: 0.4125 - val_mae: 0.5464
Epoch 8/500
10/10 [=====] - 0s 6ms/step - loss: 0.3988 - mae: 0.5287 - val_loss: 0.4060 - val_mae: 0.5452
Epoch 9/500
10/10 [=====] - 0s 5ms/step - loss: 0.3901 - mae: 0.5230 - val_loss: 0.4014 - val_mae: 0.5440
Epoch 10/500
10/10 [=====] - 0s 5ms/step - loss: 0.3804 - mae: 0.5179 - val_loss: 0.3979 - val_mae: 0.5426
Epoch 11/500
10/10 [=====] - 0s 5ms/step - loss: 0.3695 - mae: 0.5150 - val_loss: 0.3950 - val_mae: 0.5412
Epoch 12/500
10/10 [=====] - 0s 5ms/step - loss: 0.3856 - mae: 0.5245 - val_loss: 0.3921 - val_mae: 0.5399
Epoch 13/500
10/10 [=====] - 0s 6ms/step - loss: 0.3744 - mae: 0.5184 - val_loss: 0.3893 - val_mae: 0.5386
Epoch 14/500
10/10 [=====] - 0s 6ms/step - loss: 0.3749 - mae: 0.5175 - val_loss: 0.3865 - val_mae: 0.5371
Epoch 15/500
10/10 [=====] - 0s 5ms/step - loss: 0.3467 - mae: 0.4993 - val_loss: 0.3837 - val_mae: 0.5354
Epoch 16/500
```

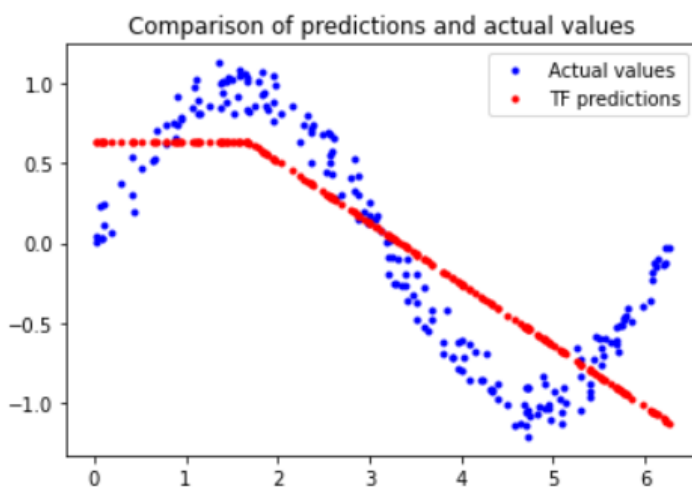
Actual vs Predicted Outputs:

To get more insight into what is happening, let's check its predictions against the test dataset we set aside earlier:


```
# Calculate and print the loss on our test dataset
test_loss, test_mae = model_1.evaluate(x_test, y_test)

# Make predictions based on our test dataset
y_test_pred = model_1.predict(x_test)

# Graph the predictions against the actual values
plt.clf()
plt.title('Comparison of predictions and actual values')
plt.plot(x_test, y_test, 'b.', label='Actual values')
plt.plot(x_test, y_test_pred, 'r.', label='TF predictions')
plt.legend()
plt.show()
```



Oh !The graph makes it clear that our network has learned to approximate the sine function in a very limited way.

The rigidity of this fit suggests that the model does not have enough capacity to learn the full complexity of the sine wave function, so it's only able to approximate it in an overly simplistic way. By making our model bigger, we should be able to improve its performance.

Training a Larger Model

1. Design the Model

To make our model bigger, let's add an additional layer of neurons. The following cell redefines our model in the same way as earlier, but with 16 neurons in the first layer and an additional layer of 16 neurons in the middle:

```
model = tf.keras.Sequential()

# First layer takes a scalar input and feeds it through 16 "neurons".
# The
# neurons decide whether to activate based on the 'relu' activation
# function.
model.add(keras.layers.Dense(16, activation='relu', input_shape=(1,)))

# The new second and third layer will help the network learn more
# complex representations
model.add(keras.layers.Dense(16, activation='relu'))

# Final layer is a single neuron, since we want to output a single
# value
model.add(keras.layers.Dense(1))

# Compile the model using the standard 'adam' optimizer and the mean
# squared error or 'mse' loss function for regression.
model.compile(optimizer='adam', loss="mse", metrics=["mae"])
```

2. Train the Model

We'll now train and save the new model.

```
# Train the model
history = model.fit(x_train, y_train, epochs=500, batch_size=64,
                    validation_data=(x_validate, y_validate))

# Save the model to disk
model.save(MODEL_TF)
```

```
Epoch 1/500
10/10 [=====] - 1s 20ms/step - loss: 0.4355 - mae: 0.5542 - val_loss: 0.4315 - val_mae: 0.5685
Epoch 2/500
10/10 [=====] - 0s 5ms/step - loss: 0.4183 - mae: 0.5548 - val_loss: 0.4157 - val_mae: 0.5581
Epoch 3/500
10/10 [=====] - 0s 6ms/step - loss: 0.3871 - mae: 0.5322 - val_loss: 0.3988 - val_mae: 0.5444
Epoch 4/500
10/10 [=====] - 0s 5ms/step - loss: 0.3954 - mae: 0.5348 - val_loss: 0.3834 - val_mae: 0.5350
Epoch 5/500
10/10 [=====] - 0s 5ms/step - loss: 0.3670 - mae: 0.5163 - val_loss: 0.3684 - val_mae: 0.5257
Epoch 6/500
10/10 [=====] - 0s 5ms/step - loss: 0.3426 - mae: 0.4999 - val_loss: 0.3532 - val_mae: 0.5166
Epoch 7/500
10/10 [=====] - 0s 5ms/step - loss: 0.3453 - mae: 0.5006 - val_loss: 0.3369 - val_mae: 0.5055
Epoch 8/500
10/10 [=====] - 0s 6ms/step - loss: 0.3182 - mae: 0.4830 - val_loss: 0.3203 - val_mae: 0.4940
Epoch 9/500
10/10 [=====] - 0s 5ms/step - loss: 0.3013 - mae: 0.4691 - val_loss: 0.3041 - val_mae: 0.4833
Epoch 10/500
10/10 [=====] - 0s 5ms/step - loss: 0.2795 - mae: 0.4523 - val_loss: 0.2867 - val_mae: 0.4699
Epoch 11/500
10/10 [=====] - 0s 20ms/step - loss: 0.2632 - mae: 0.4395 - val_loss: 0.2698 - val_mae: 0.4558
Epoch 12/500
10/10 [=====] - 0s 6ms/step - loss: 0.2581 - mae: 0.4331 - val_loss: 0.2534 - val_mae: 0.4436
Epoch 13/500
10/10 [=====] - 0s 5ms/step - loss: 0.2363 - mae: 0.4185 - val_loss: 0.2381 - val_mae: 0.4318
Epoch 14/500
```

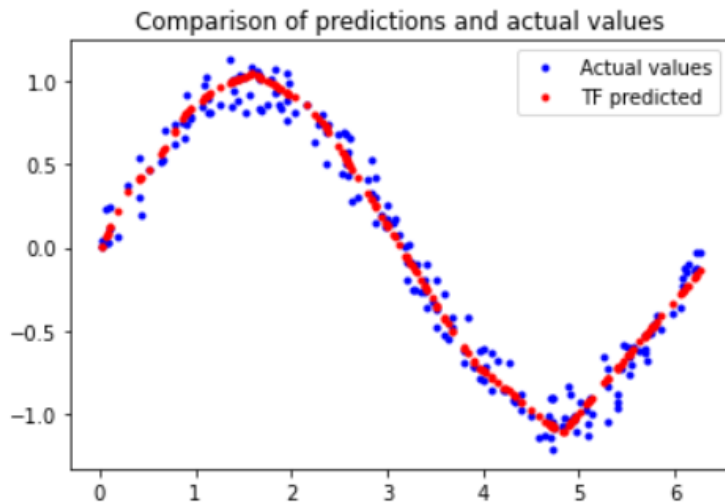
Actual vs Predicted Outputs:

To get more insight into what is happening, let's check its predictions against the test dataset we set aside earlier:

```
# Calculate and print the loss on our test dataset
test_loss, test_mae = model.evaluate(x_test, y_test)

# Make predictions based on our test dataset
y_test_pred = model.predict(x_test)

# Graph the predictions against the actual values
plt.clf()
plt.title('Comparison of predictions and actual values')
plt.plot(x_test, y_test, 'b.', label='Actual values')
plt.plot(x_test, y_test_pred, 'r.', label='TF predicted')
plt.legend()
plt.show()
```



Much better! The evaluation metrics we printed show that the model has a low loss and MAE on the test data, and the predictions line up visually with our data fairly well.

The model isn't perfect; its predictions don't form a smooth sine curve. For instance, the line is almost straight when x is between 4.2 and 5.2. If we wanted to go further, we could try further increasing the capacity of the model, perhaps using some techniques to defend from overfitting.

However, an important part of machine learning is *knowing when to stop*. This model is good enough for our use case - which is to make some LEDs blink in a pleasing pattern.

Generate a TensorFlow Lite Model

1. Generate Models with or without Quantization

We now have an acceptably accurate model. We'll use the [TensorFlow Lite Converter](#) to convert the model into a special, space-efficient format for use on memory-constrained devices.

Since this model is going to be deployed on a microcontroller, we want it to be as tiny as possible! One technique for reducing the size of a model is called [quantization](#). It reduces the precision of the model's weights, and possibly the activations (output of each layer) as well, which saves memory, often without much impact on accuracy. Quantized models also run faster, since the calculations required are simpler.

In the following cell, we'll convert the model twice: once with quantization, once without.

```
# Convert the model to the TensorFlow Lite format without quantization
converter = tf.lite.TFLiteConverter.from_saved_model(MODEL_TF)
model_no_quant_tflite = converter.convert()

# Save the model to disk
open(MODEL_NO_QUANT_TFLITE, "wb").write(model_no_quant_tflite)

# Convert the model to the TensorFlow Lite format with quantization
def representative_dataset():
    for i in range(500):
        yield([x_train[i].reshape(1, 1)])
# Set the optimization flag.
converter.optimizations = [tf.lite.Optimize.DEFAULT]
# Enforce integer only quantization
converter.target_spec.supported_ops =
[tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
# Provide a representative dataset to ensure we quantize correctly.
converter.representative_dataset = representative_dataset
model_tflite = converter.convert()

# Save the model to disk
open(MODEL_TFLITE, "wb").write(model_tflite)
```

Compare Model Performance

To prove these models are accurate even after conversion and quantization, we'll compare their predictions and loss on our test dataset.

```
def predict_tflite(tflite_model, x_test):
    # Prepare the test data
    x_test_ = x_test.copy()
    x_test_ = x_test_.reshape((x_test.size, 1))
    x_test_ = x_test_.astype(np.float32)

    # Initialize the TFLite interpreter
    interpreter = tf.lite.Interpreter(model_content=tflite_model,
                                     experimental_op_resolver_type=tf.li
te.experimental.OpResolverType.BUILTIN_REF)
    interpreter.allocate_tensors()

    input_details = interpreter.get_input_details()[0]
    output_details = interpreter.get_output_details()[0]
```



विद्याधनं सर्वधनं प्रधानम्

IIT JAMMU

```
# If required, quantize the input layer (from float to integer)
input_scale, input_zero_point = input_details["quantization"]
if (input_scale, input_zero_point) != (0.0, 0):
    x_test_ = x_test_ / input_scale + input_zero_point
    x_test_ = x_test_.astype(input_details["dtype"])

# Invoke the interpreter
y_pred = np.empty(x_test_.size, dtype=output_details["dtype"])
for i in range(len(x_test_)):
    interpreter.set_tensor(input_details["index"], [x_test_[i]])
    interpreter.invoke()
    y_pred[i] = interpreter.get_tensor(output_details["index"])[0]

# If required, dequantized the output layer (from integer to float)
output_scale, output_zero_point = output_details["quantization"]
if (output_scale, output_zero_point) != (0.0, 0):
    y_pred = y_pred.astype(np.float32)
    y_pred = (y_pred - output_zero_point) * output_scale

return y_pred

def evaluate_tflite(tflite_model, x_test, y_true):
    global model
    y_pred = predict_tflite(tflite_model, x_test)
    loss_function = tf.keras.losses.get(model.loss)
    loss = loss_function(y_true, y_pred).numpy()
    return loss
```

Predictions

- # Calculate predictions
- y_test_pred_tf = model.predict(x_test)
- y_test_pred_no_quant_tflite = predict_tflite(model_no_quant_tflite, x_test)
- y_test_pred_tflite = predict_tflite(model_tflite, x_test)

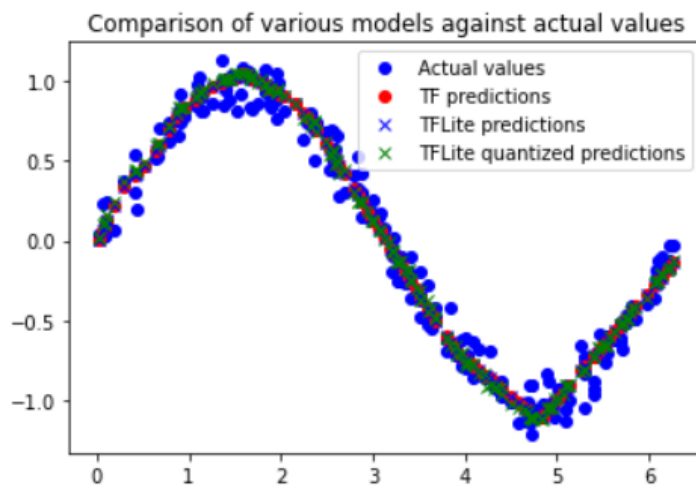
```
# Compare predictions
plt.clf()
plt.title('Comparison of various models against actual values')
plt.plot(x_test, y_test, 'bo', label='Actual values')
```



विद्याधनं सर्वधनं प्रधानम्

IIT JAMMU

```
plt.plot(x_test, y_test_pred_tf, 'ro', label='TF predictions')
plt.plot(x_test, y_test_pred_no_quant_tflite, 'bx', label='TFLite
predictions')
plt.plot(x_test, y_test_pred_tflite, 'gx', label='TFLite quantized
predictions')
plt.legend()
plt.show()
```



Loss (MSE/Mean Squared Error)

```
# Calculate loss
loss_tf, _ = model.evaluate(x_test, y_test, verbose=0)
loss_no_quant_tflite = evaluate_tflite(model_no_quant_tflite, x_test,
y_test)
loss_tflite = evaluate_tflite(model_tflite, x_test, y_test)
# Compare loss
df = pd.DataFrame.from_records(
    [
        ["TensorFlow", loss_tf],
        ["TensorFlow Lite", loss_no_quant_tflite],
        ["TensorFlow Lite Quantized", loss_tflite]
    ],
    columns = ["Model", "Loss/MSE"], index="Model").round(4)
df
```

Loss/MSE	
Model	
TensorFlow	0.0102
TensorFlow Lite	0.0102
TensorFlow Lite Quantized	0.0108

Generate a TensorFlow Lite for Microcontrollers Model

Convert the TensorFlow Lite quantized model into a C source file that can be loaded by TensorFlow Lite for Microcontrollers.

```
# Install xxd if it is not available
!apt-get update && apt-get -qq install xxd
# Convert to a C source file, i.e, a TensorFlow Lite for
Microcontrollers model
!xxd -i {MODEL_TFLITE} > {MODEL_TFLITE_MICRO}
# Update variable names
REPLACE_TEXT = MODEL_TFLITE.replace('/', '_').replace('.', '_')
!sed -i 's/{REPLACE_TEXT}/g_model/g' {MODEL_TFLITE_MICRO}
```

Deploy to a Microcontroller

Follow the instructions in the [hello_world](#) README.md for [TensorFlow Lite for Microcontrollers](#) to deploy this model on a specific microcontroller.

```
# Print the C source file
!cat {MODEL_TFLITE_MICRO}
0x04, 0x00, 0x00, 0x00, 0x76, 0xfd, 0xff, 0xff, 0x04, 0x00, 0x00, 0x00,
0x0d, 0x00, 0x00, 0x00, 0x64, 0x65, 0x6e, 0x73, 0x65, 0x5f, 0x32, 0x5f,
0x69, 0x6e, 0x70, 0x75, 0x74, 0x00, 0x00, 0x00, 0x01, 0x00, 0x00, 0x00,
0x0c, 0x00, 0x00, 0x00, 0x08, 0x00, 0x0c, 0x00, 0x08, 0x00, 0x04, 0x00,
0x08, 0x00, 0x00, 0x00, 0x0b, 0x00, 0x00, 0x00, 0x04, 0x00, 0x00, 0x00,
0x13, 0x00, 0x00, 0x00, 0x6d, 0x69, 0x6e, 0x5f, 0x72, 0x75, 0x6e, 0x74,
0x69, 0x6d, 0x65, 0x5f, 0x76, 0x65, 0x72, 0x73, 0x69, 0x6f, 0x6e, 0x00,
0x0c, 0x00, 0x00, 0x00, 0x50, 0x02, 0x00, 0x00, 0x48, 0x02, 0x00, 0x00,
0x34, 0x02, 0x00, 0x00, 0xdc, 0x01, 0x00, 0x00, 0x8c, 0x01, 0x00, 0x00,
0x6c, 0x01, 0x00, 0x00, 0x5c, 0x00, 0x00, 0x00, 0x3c, 0x00, 0x00, 0x00,
0x34, 0x00, 0x00, 0x00, 0x2c, 0x00, 0x00, 0x00, 0x24, 0x00, 0x00, 0x00,
0x04, 0x00, 0x00, 0x00, 0xfa, 0xfd, 0xff, 0xff, 0x04, 0x00, 0x00, 0x00,
0x10, 0x00, 0x00, 0x00, 0x31, 0x2e, 0x35, 0x2e, 0x30, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x84, 0xfd, 0xff, 0xff,
0x88, 0xfd, 0xff, 0xff, 0x8c, 0xfd, 0xff, 0xff, 0x22, 0xfe, 0xff, 0xff,
0x04, 0x00, 0x00, 0x00, 0x10, 0x00, 0x00, 0x00, 0x21, 0xa5, 0x8b, 0xca,
0x5e, 0x1d, 0xce, 0x42, 0x9d, 0xce, 0x1f, 0xb0, 0xdf, 0x54, 0x2f, 0x81,
0x3e, 0xfe, 0xff, 0xff, 0x04, 0x00, 0x00, 0x00, 0x00, 0x01, 0x00, 0x00,
0xee, 0xfc, 0x00, 0xec, 0x05, 0x17, 0xef, 0xec, 0xe6, 0xf8, 0x03, 0x01,
0x00, 0xfa, 0xf8, 0xf5, 0xdc, 0xeb, 0x27, 0x14, 0xf1, 0xde, 0xe2, 0xdb,
0xf0, 0xde, 0x31, 0x06, 0x02, 0xe6, 0xee, 0xf9, 0x00, 0x16, 0x07, 0xe0,
0xfe, 0xff, 0xe9, 0x06, 0xe7, 0xef, 0x81, 0x1b, 0x18, 0xea, 0xc9, 0x01,
0x0f, 0x00, 0xda, 0xf7, 0x0e, 0xec, 0x13, 0x1f, 0x04, 0x13, 0xb4, 0xe6,
0xfd, 0x06, 0xb9, 0xe0, 0x0d, 0xec, 0xf0, 0xde, 0xeb, 0xf7, 0x05, 0x26,
```


Here is an code of the sine wave generation might look:([Harvard TinyMLx—Hello world](#))

```
#include <TensorFlowLite.h>

#include "main_functions.h"

#include "tensorflow/lite/micro/all_ops_resolver.h"
#include "constants.h"
#include "model.h"
#include "output_handler.h"
#include "tensorflow/lite/micro/micro_error_reporter.h"
#include "tensorflow/lite/micro/micro_interpreter.h"
#include "tensorflow/lite/schema/schema_generated.h"
#include "tensorflow/lite/version.h"

// Globals, used for compatibility with Arduino-style sketches.
namespace {
  tflite::ErrorReporter* error_reporter = nullptr;
  const tflite::Model* model = nullptr;
  tflite::MicroInterpreter* interpreter = nullptr;
  TfLiteTensor* input = nullptr;
  TfLiteTensor* output = nullptr;
  int inference_count = 0;

  constexpr int kTensorArenaSize = 2000;
  uint8_t tensor_arena[kTensorArenaSize];
} // namespace

// The name of this function is important for Arduino compatibility.
void setup() {
  // Set up logging. Google style is to avoid globals or statics because of
  // lifetime uncertainty, but since this has a trivial destructor it's okay.
  // NOLINTNEXTLINE(runtime-global-variables)
  static tflite::MicroErrorReporter micro_error_reporter;
  error_reporter = &micro_error_reporter;

  // Map the model into a usable data structure. This doesn't involve any
  // copying or parsing, it's a very lightweight operation.
  model = tflite::GetModel(g_model);
  if (model->version() != TFLITE_SCHEMA_VERSION) {
    TF_LITE_REPORT_ERROR(error_reporter,
                        "Model provided is schema version %d not equal "
```

```

        "to supported version %d.",
        model->version(), TFLITE_SCHEMA_VERSION);

    return;
}

// This pulls in all the operation implementations we need.
// NOLINTNEXTLINE(runtime-global-variables)
static tflite::AllOpsResolver resolver;

// Build an interpreter to run the model with.
static tflite::MicroInterpreter static_interpreter(
    model, resolver, tensor_arena, kTensorArenaSize, error_reporter);
interpreter = &static_interpreter;

// Allocate memory from the tensor_arena for the model's tensors.
TfLiteStatus allocate_status = interpreter->AllocateTensors();
if (allocate_status != kTfLiteOk) {
    TF_LITE_REPORT_ERROR(error_reporter, "AllocateTensors() failed");
    return;
}

// Obtain pointers to the model's input and output tensors.
input = interpreter->input(0);
output = interpreter->output(0);

// Keep track of how many inferences we have performed.
inference_count = 0;
}

// The name of this function is important for Arduino compatibility.
void loop() {
    // Calculate an x value to feed into the model. We compare the current
    // inference_count to the number of inferences per cycle to determine
    // our position within the range of possible x values the model was
    // trained on, and use this to calculate a value.
    float position = static_cast<float>(inference_count) /
        static_cast<float>(kInferencesPerCycle);
    float x = position * kXrange;

    // Quantize the input from floating-point to integer
    int8_t x_quantized = x / input->params.scale + input->params.zero_point;
    // Place the quantized input in the model's input tensor
    input->data.int8[0] = x_quantized;

```



विद्याधनं सर्वधनं प्रधानम्

IIT JAMMU

```
// Run inference, and report any error
TfLiteStatus invoke_status = interpreter->Invoke();
if (invoke_status != kTfLiteOk) {
    TF_LITE_REPORT_ERROR(error_reporter, "Invoke failed on x: %f\n",
                          static_cast<double>(x));
    return;
}

// Obtain the quantized output from model's output tensor
int8_t y_quantized = output->data.int8[0];
// Dequantize the output from integer to floating-point
float y = (y_quantized - output->params.zero_point) * output->params.scale;

// Output the results. A custom HandleOutput function can be implemented
// for each supported hardware target.
HandleOutput(error_reporter, x, y);

// Increment the inference_counter, and reset it if we have reached
// the total number per cycle
inference_count += 1;
if (inference_count >= kInferencesPerCycle) inference_count = 0;
}
```

Visualizing Data Using Serial Plotter

1. Setup:

- Open the Arduino IDE and upload the sketch to the Arduino Nano 33 BLE Lite.
- Ensure the Serial Plotter is available by navigating to Tools > Serial Plotter in the Arduino IDE.

2. Visualization:

- The Serial Plotter will display real-time graphical representation of the sine wave data sent from the Arduino.
- As the data points are transmitted serially, the plot updates dynamically to reflect the oscillatory pattern of the sine wave.

Summary:

- The sine wave data was generated by calculating the sine values for a range of angles and then transmitting these values to the Serial Plotter.

Data Transfer: From Arduino to DAC Using I2C

1. Overview

The process of transferring data to the DAC (Digital-to-Analog Converter) using I2C and converting it to an analog signal involves several key steps: initializing the I2C communication, sending data to the DAC, and ensuring proper data conversion and output. Here's a detailed breakdown:

2. I2C Communication Setup

I2C Initialization:

- **Library Inclusion:** Ensure that the Wire library, which handles I2C communication, is included in your Arduino sketch. This library provides the necessary functions for communication between the Arduino and I2C devices.
- **Start I2C:** Initialize I2C communication in the `setup()` function using `Wire.begin()`. This function configures the Arduino as an I2C master.

3. Data Transmission to the DAC

Addressing the DAC:

- **DAC Address:** Identify the I2C address of the DAC DFR0972. This address is specified in the DAC's datasheet or documentation. For this example, we'll assume the DAC address is 0x58.

Sending Data:

- **Prepare Data:** Convert the sine wave data to a format suitable for the DAC. Most DACs accept data in a specific resolution (e.g., 12-bit), so scale the sine wave values accordingly.
- **Write Data:** Use the `Wire.beginTransmission()` function to start communication with the DAC, followed by `Wire.write()` to send the data, and `Wire.endTransmission()` to end the communication.

Here's a complete code for integrating data generation, transmission, and DAC output:

```
#include <Wire.h>

const int numPoints = 100;

const float pi = 3.14159;

float sineWave[numPoints];

const int dacAddress = 0x58; // DAC I2C address

void setup() {

    Wire.begin();

    generateSineWave();

}

void loop() {

    for (int i = 0; i < numPoints; i++) {

        uint16_t dacValue = (sineWave[i] + 1.0) * 2047; // Convert to 12-bit value

        sendDataToDAC(dacValue);
```

```
delay(50); // Delay to simulate waveform frequency
```

```
}
```

```
}
```

```
void generateSineWave() {
```

```
    for (int i = 0; i < numPoints; i++) {
```

```
        float angle = (2 * pi / numPoints) * i;
```

```
        sineWave[i] = sin(angle);
```

```
    }
```

```
}
```

```
void sendDataToDAC(uint16_t value) {
```

```
    Wire.beginTransaction(dacAddress);
```

```
    Wire.write((value >> 8) & 0xFF); // High byte
```

```
    Wire.write(value & 0xFF);      // Low byte
```

```
    Wire.endTransmission();
```

```
}
```

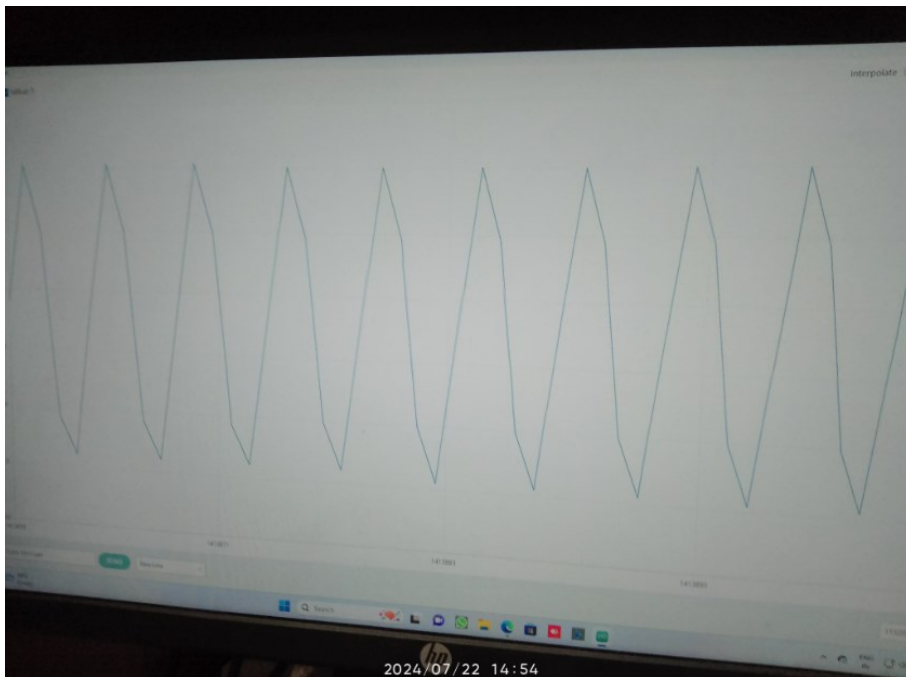
This process allows for real-time generation and conversion of sine wave data, making it suitable for applications requiring analog signal outputs from digital sources.

4.Results and Discussion

Generated Sine Waves:

Data Presentation:

- **Serial Plotter:** The sine wave data generated by the Arduino Nano 33 BLE Lite was visualized using the Serial Plotter. The plot displayed a smooth, periodic oscillation, characteristic of a sine wave. Below is an example screenshot of the sine wave as seen on the Serial Plotter:

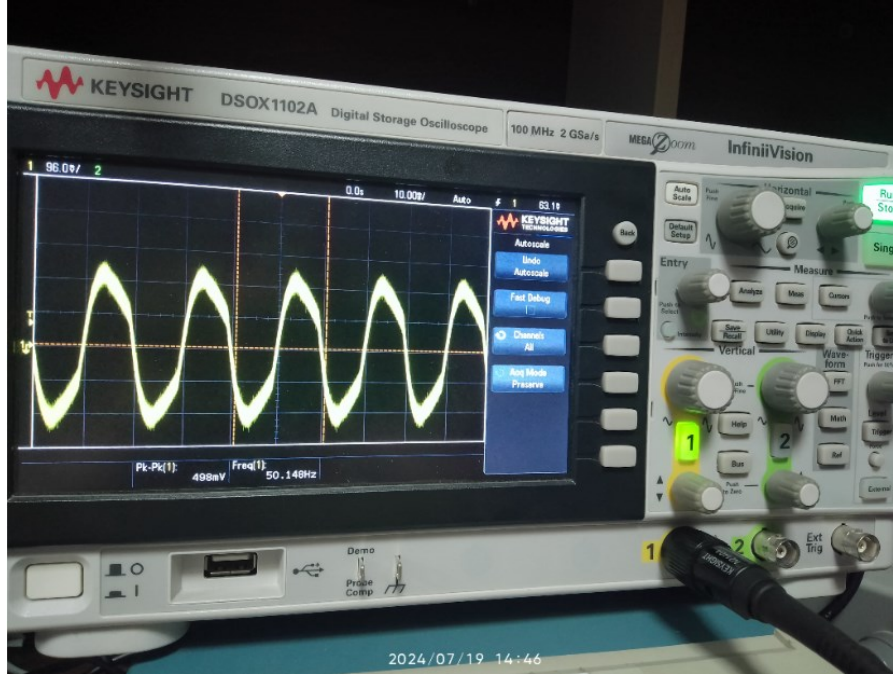


- **Oscilloscope:** The same sine wave data was transmitted to the DAC DFR0972 and visualized on an oscilloscope. The oscilloscope showed a clean, continuous sine wave signal, demonstrating the DAC's capability to accurately convert digital data into analog form. Below is an example image of the sine wave displayed on the oscilloscope:



विद्याधनं सर्वधनं प्रधानम्

IIT JAMMU



5. Conclusion

Summary of Findings

The project successfully demonstrated the generation and visualization of sine wave data using the Arduino Nano 33 BLE Lite and the DAC DFR0972. Key findings include:

1. **Sine Wave Generation:** The Arduino was able to generate smooth and accurate sine wave data using the mathematical sine function $y = \sin(x)$. This data was transmitted to the DAC, which converted it into an analog signal.
2. **Visualization:** The generated sine wave was effectively visualized using the Arduino Serial Plotter and an oscilloscope. The Serial Plotter displayed the sine wave in real-time, while the oscilloscope confirmed the analog signal's accuracy and fidelity.
3. **Data Accuracy:** The analysis showed that the sine wave data closely matched the theoretical expectations. The amplitude and frequency were consistent with the intended values, indicating that both the data generation and conversion processes were accurate.

Overall, the project validated the integration of TinyML concepts with embedded systems for real-time signal processing and demonstrated the practical application of microcontrollers and DACs in generating analog signals.

Future Work

1. **Refinement of Sine Wave Generation:**
 - **Higher Resolution:** Explore the use of higher-resolution DACs to improve the accuracy and smoothness of the generated sine wave.
 - **Dynamic Frequency and Amplitude:** Implement functionality to dynamically adjust the frequency and amplitude of the sine wave, allowing for more flexible signal generation.
2. **Advanced TinyML Applications:**



- **Complex Signal Processing:** Investigate more complex signal processing tasks using TinyML, such as real-time filtering, modulation, or Fourier analysis.
- **Machine Learning Models:** Develop and deploy machine learning models on the Arduino Nano 33 BLE Lite for tasks such as pattern recognition or predictive analytics based on sensor data.

This project provides a solid foundation for further exploration and development in the field of TinyML and embedded systems, offering numerous opportunities for enhancement and application in diverse scenarios.

6. Project Summary

Objective: The objective of the project was to generate sine wave signals using the Arduino Nano 33 BLE Lite and visualize these signals on an oscilloscope, while exploring the integration of TinyML for optimizing the wave generation process.

1. Sine Wave Data Generation

- The project began with generating sine wave data using the formula $Y = \sin(x)$. This was programmed into the Arduino, where the x-values were incrementally increased to produce a smooth sine wave pattern.
- The generated sine wave data was visualized using the Serial Plotter feature in the Arduino IDE. This step was crucial for verifying the accuracy of the waveform and ensuring the correct implementation of the sine function.

Steps:

1. Model Selection
2. Training the Model
3. Deployment and Real-Time Adjustment
4. Upload the data on Arduino
5. Find results on serial plotter

2. Data Transfer to DAC



विद्याधनं सर्वधनं प्रधानम्

IIT JAMMU

- The generated sine wave data was then transmitted from the Arduino to the DAC DFR0972 via I2C communication. This process involved setting up the correct I2C addresses and ensuring the data was formatted correctly for transmission.
- Once received by the DAC, the digital sine wave data was converted into an analog signal. This conversion was essential for visualizing the waveform on an oscilloscope, where the sine wave could be observed in real time.

3. Visualization and Optimization

- The analog signal produced by the DAC was displayed on an oscilloscope, allowing for a detailed examination of the sine wave. Adjustments were made to the waveform parameters to achieve the desired output, focusing on aspects like amplitude and frequency.

4. Challenges and Solutions:

Throughout the project, various challenges were encountered, such as ensuring accurate data transfer between the Arduino and DAC and fine-tuning the waveform's shape and quality. These issues were addressed through iterative testing and adjustments to the code and hardware setup.

The project successfully demonstrated the ability to generate, transfer, and visualize sine waves using the Arduino Nano 33 BLE Lite and DAC DFR0972.

7. References

1. Books:

- Warden, P., & Situnayake, D. (2019). *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. O'Reilly Media.

2. Datasheets:

- Arduino Nano 33 BLE Lite , [Download](#)
- DAC DFR0972: DFRobot. [Download](#)

3. Tools and Software:

- Arduino IDE
- Libraries

This list includes the primary references used in the preparation of the project, covering essential books, datasheets, and tools relevant to the work on sine wave generation and TinyML applications.